

CENG444 - Language Processors

2025-2026 Spring

Project IV**Simple Optimizations, Linear IR, and Code Generation**

Problem

In this assignment, you are required to write a C++ program that generates linear IR after performing simple optimizations. Your implementation will be based on your Project III implementation. There is no change in the language specification.

Constant folding

Your solution will have the capacity to reduce the code requirement to evaluate the expressions based on the roles of constants found in an expression. The result that is going to be evaluated by an operator can be evaluated ahead of time by using constant folding. Below is the list of the operators and constant dependent contexts where constant folding can be applied and how.

Ternary Operator

```
true?<expr1>:<expr2> => <expr1>
false?<expr1>:<expr2> => <expr2>
```

Comparison Operators

```
const <comp-op> const => either true or false
```

Boolean and Operator

```
<expr> && true => <expr>
true && <expr> => <expr>
<expr> && false=> false, when <expr> is ineffective
false && <expr> => false, when <expr> is ineffective
```

Boolean or Operator

```
<expr> || false => <expr>
false || <expr> => <expr>
<expr> || true => true, when <expr> is ineffective
true || <expr> => true, when <expr> is ineffective
```

Addition / concatenation operator

```
<const> + <const> => <const>
<expr> + 0 => <expr>
0 + <expr> => <expr>
<expr> + "" => <expr>
"" + <expr> => <expr>
```

Subtraction Operator

```
<const> - <const> => <const>
<expr> - 0 => <expr>
0 - <expr> => -<expr>
```

Multiplication Operator

```
<const> * <const> => <const>
<expr> * 0 => 0, when <expr> is ineffective
0 * <expr> => 0, when <expr> is ineffective
```

Division Operator

```
<const> / 0 => 3 with an error message. 3 is an ordinary value
that does not trigger further constant folding.
<const> / <const> => <const>
0 / <expr> => 0, when <expr> is ineffective
<expr> / 1 => <expr>
```

Unary Minus Operator

```
- number => -number
```

The Not Operator

```
! const => !const
```

Dead code elimination

Ineffective Statements (-p1 / OPTIMIZE_DC_NONSELECT)

A where clause in a select statement can be found always false after applying constant folding. In this case, the scanner loops, the aggregation and snapshot codes can be skipped altogether. This optimization will not be applied if the where condition has at least one select statement.

Ineffective Expression Parts (-p2 / OPTIMIZE_DC_EXPPART)

Another form of a dead code elimination may occur where an expression with comma operator(s) found out of call contexts. In such cases, an expression part is said to be effective when any combinations of the following conditions are met:

- If the expression part is the first in the list.
- If the expression has any call to a function having “forceActive” property set. The print function¹ is an example.

Similarly, there is no need to execute an ineffective expression part. Even when an expression part is found as ineffective, the graphical IR will contain fully analyzed expression nodes. However, the IC will not contain any code from the ineffective expression parts.

¹ This may help you debug.

Invariant code motion

Dynamically Calculatable Fixed Case (-p4 / OPTIMIZE_CM_DYNAFIX)

An expression is said to be idempotent when it does not contain a select and all of the function calls are idempotent. For the idempotent where-expressions that cannot be calculated statically (at compile time) the execution template of a select statement can be optimized to avoid constant calculations.

IC Generation

Your program will generate intermediate code and present it both in a JSON file, which becomes the inspection tool for the intermediate representation, and in a human readable text file.² An instruction will have a sequence number, op-code, a simple type-tag, and an optional parameter. See Appendix 2: Node Op-Codes and IC Instruction Set for a concise description of the instruction set.

The linear IC that will be generated for a stack machine that will be discussed in the lab hours with help of example code fragments.

Most of the instructions correspond to the operators defined in the supplemental code. A few of the expression tree patterns are translated compiler runtime calls. Examples are:

- String concatenations that appear as OP_ADD with array operands.
- Comparison op-codes with string parameters.

There is a small set of IC instructions that neither appear as direct op-codes in expression nodes nor translated to compiler runtime calls. These are the instructions to perform jumps and stack manipulation.

Peephole optimization

The peephole optimization that will be applied by your solution will have a two-instruction window just to identify a single kind of ineffective code. You are given helper methods to implement peephole optimization having a limited window size.

Trivial Jump Elimination (-p8 / OPTIMIZE_PH_TRIVIALJUMP)

Any instruction sequence having the following patterns will be eliminated.

Pattern 1:

```
const true      or,   const false
jf <a target>    jt <a target>
```

Pattern 2:

```
                jmp L1
L1:
```

² IC samples given in the ODTU Class Project 4 pages contain plenty of instruction lines to give you an idea on how the human readable text file looks like.

Conditional jumps that can be evaluated ahead of time can be reduced to unconditional jumps.

Pattern 3:

<code>const false</code>	<code>or,</code>	<code>const true</code>
<code>jf <a target></code>		<code>jt <a target></code>
<code>transforms to</code>		<code>transforms to</code>
<code>jmp <a target></code>		<code>jmp <a target></code>

See the ic-samples containing two sample rml compiled under various optimization flags in the project discussion forum for this assignment on relevant ODTU Class pages.

X64 Code Generation (Bonus)

Your solution will translate the intermediate code into x64 instructions into an executable memory block and execute it. The code templates and methodologic guidance will be provided in the lab hours with additional code support.

Important Note!

In contrary to the previous assignment, a “where condition” may involve multiple top level comma operators which lead production of more than one value.

Your Assignment

You will develop a C++ program that accepts one or two command line arguments. When run with single argument, your program will treat the parameter as the file name. When run with two parameters, the first parameter will be in form of -p<number> where the number is the bitwise encoded optimization flag. In this case, the second parameter will become the file name.

The file name parameter will be used to generate the full name of the input file and the output files. The input file will be <file name>.txt, the first output file will be <file name>.json, and the second output file will be <file name>-IC.txt. The first output file having the JSON format will be similar to the output of the Project III implementation. The second output file will be a text file that contains easy to read IC listing.

When specified, the bitwise specified optimization flag will tell your program the optimizations that will be applied while generating the IC.

Argument	Macro in rmlsup.h	Meaning
-p1	OPTIMIZE_DC_NONSELECT	The processor will apply dead code elimination on where conditions that calculate statically false.
-p2	OPTIMIZE_DC_EXPPART	The processor will apply dead code elimination on ineffective expression parts.
-p4	OPTIMIZE_CM_DYNAFIX	The processor will change the select loop template when applicable.

Argument	Macro in rmlsup.h	Meaning
-p8	OPTIMIZE_PH_TRIVIALJUMP	The processor will eliminate trivial conditional jumps that follows a Boolean constant push.

Note that the bitwise encoded arguments must be combined to activate more than one optimization. Specifying -p15 will activate all of the optimizations. Specifying -p0 will disable all of the optimizations. Your program will apply all kind of optimizations when run in single parameter configuration.

Example run:

When your program is compiled and run as “project4 test1”, your program will read the input file “test1.txt”, and generate the outputs as “test1.json” and “test1-IC.txt”. In case your program fails to open the input file, your program will print “File not found!” message and terminate.

Processor Output

The JSON file will meet all of the requirements of the Project III. The followings define additional or modified entities that will be found:

- Following counts must be reported in each node of the expression trees.
 - Number of the function calls in the subtree.
 - Number of the non-idempotent function calls in the subtree.
 - Number of the detected column references in the subtree.
 - Number of the selects found in the subtree.
 - Number of the function calls having active force set in the subtree.
- There will be a root level array containing the IC instructions. Instruction reporting format will be supplied. There will be no instruction coming from an eliminated statement.
- There will be mutations and transformations on the expression trees as result of processing. The final version of the expression trees must be observed in the JSON file.

The output text file will contain instruction lines. Each instruction line will contain sequence number, instruction mnemonic, type and parameter information as appropriate. The example format for an instruction line will be supplied.

Finally, the processor must report to the console the messages placed in the JSON file in a human readable format. This report must be generated before calling the dynamically generated x64 code if bonus part was implemented.

Bonus Part Output

Those who intends two receive bonus points must note in the README.txt file their intention. For the submissions with bonus part, the program is expected to run the dynamically generated code after generating the standard processor outputs. Note that execution of dynamically generated code is requested for error free RML code only. It is student’s responsibility to implement the RML runtime library functions (see Appendix 1: RML Runtime Library). Further guidance will be provided in implementation of these functions during the lab hours.

The evaluation process will use the print function to observe correct execution. The README.txt file must provide the evaluator where to set a breakpoint and the variable that points to the memory block that contains the dynamically generated x64 code.

Regulations & Hints

- **Implementation:** You should use Flex, Bison with C++ to develop your program. Make sure that your program will accept the name of the input file. In case the program is run without a parameter or with more than one parameter your program must terminate immediately with appropriate error message sent to the standard output.
- **Supplementary Material:** You will be provided with a supplemental code and an example set for support and standardization. The code will cover
 - A standardized grammar for RMLEval
 - A small library for fundamental data structures, runtime functions, and the language runtime.

Note that you may be asked to develop your own extensions to any of these components.

- **Evaluation:** The evaluation will be based on
 - Presence and correctness of the symbol table in the JSON file when the input is free of syntactic problems.
 - Presence and correctness of the statement lists in the JSON file for both circular and drafted ones when the input is free of syntactic problems.
 - Presence and correctness of the expression trees in the JSON file for each statement when the input is free of syntactic problems.
 - Presence and the correctness of the linear IR both in the JSON file and in the more human readable text file.
 - Factual presence of the messages in the JSON file due to syntactic and semantic checks with line number and descriptions.

Keep in mind that correctness checks will also require the coherence between the IC generated and the optimization flags specified.

- **Submission:** You need to submit all relevant files you have implemented (.cpp, .h, .l, .y, etc.) as well as a README file with instructions on how to build and run, in a single .zip file named <your studentID>.

Following notes are for more clarity to address possible concerns and/or questions:

- Use Flex and Bison to generate C++ not C file. C implementation will not be accepted.
- Never modify the automatically generated files generated by Flex and Bison. You may prefer consulting the recitation material and the sample project on semantic analysis.
- Describe precisely the steps to build and run your program in README.txt file, which is essential part of your submission. Provide any configuration items such as scripts, configuration files that will be necessary.

Appendix 1: RML Runtime Library

RML runtime library includes a limited set of functions that can be accessed by using function call operator. RML, has two types of functions:

- Aggregate functions (A):

These functions can be used in column lists only. An aggregate function in a column, calculates a single value by using each eligible column found in the *rowset* generated by the select operator which the column belongs to.

- Scalar functions (S):

A scalar function generates does not have limitation of aggregate functions. It operates on its parameters and returns a value.

Index	Prototype	A/S	Description
0	<code>number stddev(numbers)</code>	A	Calculates standard deviation of the values calculated for each of the rows
1	<code>number mean(numbers)</code>	A	Calculates the mean of the values calculated for each of the rows.
2	<code>number count()</code>	A	Calculates row count.
3	<code>number min(numbers)</code>	A	Calculates the minimum of the values calculated for each of the rows.
4	<code>number max(numbers)</code>	A	Calculates the maximum of the values calculated for each of the rows.
5	<code>number sum(numbers)</code>	A	Calculates the sum of the values calculated for each of the rows.
6	<code>number sin(number)</code>	S	Calculates the sine of a given number.
7	<code>number cos(number)</code>	S	Calculates the cosine of a given number.
8	<code>number tan(number)</code>	S	Calculates the tangent of a given number.
9	<code>number pi()</code>	S	Returns pi.
10	<code>number atan(number)</code>	S	Calculates the inverse tangent of a given number.
11	<code>number asin(number)</code>	S	Calculates the inverse sine of a given number.
12	<code>number acos(number)</code>	S	Calculates the inverse cosine of a given number.
13	<code>number exp(number)</code>	S	Evaluates e powered to the given parameter.
14	<code>number ln(number)</code>	S	Calculates natural logarithm of the given parameter.
15	<code>number number(string)</code>	S	Converts a string to the corresponding number.
16	<code>number random(number)</code>	S	Returns a random number x for a given number y (y>0) with the following condition: 0 <= x < y.
17	<code>number print(string)</code>	S	Sends the parameter to the standard console. Returns 1.

Index	Prototype	A/S	Description
18	number len(string)	S	Returns the length of a given string.
19	string right(string, n)	S	Returns a new string containing rightmost n characters of the first parameter.
20	string left(string, n)	S	Returns a new string containing leftmost n characters of the first parameter.

Appendix 2: Node Op-Codes and IC Instruction Set

Below is the table of the op-codes that can be found in an expression tree and their corresponding mnemonics and semantics.

Op Code	mnemonic	Functional Description
OP_NOP	nop	Does nothing.
OP_COMMA	comma	Accumulates expression value on top of stack. The newly evaluated value remains as word on TOS.
OP_COND	cond	Appears in expression trees to denote the conditional evaluation (ternary) operator. This is not an IC instruction.
OP_ALT	alt	Appears in expression trees to denote the alternatives for conditional evaluation (ternary) operator. This is not an IC instruction.
OP_BAND	band	Applies logical and to the two words at the top of the stack. It effectively pops these two values and pushes the result.
OP_BOR	bor	Applies logical or to the two words at the top of the stack. It effectively pops these two values and pushes the result.
OP_EQ	eq	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if both operands are equal, false otherwise. This op-code must be replaced by a proper LRT instruction by the processing subsequent to the initial expression tree construction for the operations involving strings and arrays.
OP_NEQ	neq	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if both operands are not equal, false otherwise. This op-code must be replaced by a proper LRT instruction by the processing subsequent to the initial expression tree construction for the operations involving strings and arrays.
OP_LT	lt	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if the first operand is less than the second, false otherwise.
OP_LTE	lte	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if the first operand is less than the second operand or equal to the second operand, false otherwise.
OP_GT	gt	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if the first operand is greater than the second, false otherwise.
OP_GTE	gte	Compares two words at the top of the stack. It effectively pops these two values and pushes the result. The result is logical true if the first operand is greater than the second operand or equal to the second operand, false otherwise.
OP_ADD	add	Adds two words at the top of the stack. It effectively pops these two values and pushes the result. This op-code must be replaced by a proper LRT instruction by the processing subsequent to the initial expression tree construction for the operations involving strings and arrays.
OP_SUB	sub	Subtracts the word at the top of the stack from the word that was pushed before the word appearing at the top of the stack. It effectively pops two values and pushes the result.
OP_MUL	mul	Multiplies the word at the top of the stack by the word that was pushed before the word appearing at the top of the stack. It effectively pops two values and pushes the result.
OP_DIV	div	Divides the word that was pushed before the word appearing at the top of the stack by the word at the top of the stack. It effectively pops two values and pushes the result.
OP_MINUS	minus	Changes the value of the word appearing at the top of the stack as negation of its current value. Stack load does not change.
OP_NOT	not	Changes the value of the word appearing at the top of the stack as logical negation of its current value. Stack load does not change.

Op Code	mnemonic	Functional Description
OP_CALL	call	Calls the function having the registration index number stored in the numConstant member. The p1 parameter contains number of the actual parameters.
JMP	jmp	Performs jump to the linear IR code location pointed by the p1 member.
JF	jf	When the value of the word at the top of the stack is logical false, this instruction performs a jump to the linear IR code location pointed by the p1 member. This instruction pops the word inspected for the logical check.
JT	jt	When the value of the word at the top of the stack is logical true, this instruction performs a jump to the linear IR code location pointed by the p1 member. This instruction pops the word inspected for the logical check.
SYMREF	symref	Pushes the value of the word implied by the p1 member onto the stack.
CONST	const	Pushes the immediate value stored in the instruction onto the stack.
LRT	lrt	Performs a call to the language runtime function implied by the p1 member, which must be a valid index to an LRT function. Below is the list of additional critical members in an LRT instruction that need to be generated. • LRT_RESETSCANNER must have the space id encoded in member intConstant. • LRT_ADVANCESCANNER must have the space id encoded in member intConstant. • LRT_AGGREGATE must have the space id encoded in member intConstant. • LRT_SNAPSHOT must have the space id encoded in member intConstant. • LRT_CREATERESULTSET must have the space id encoded in member intConstant. • LRT_FINALIZERRESULTSET must have the space id encoded in member intConstant. • LRT_STRCMP must have the comparison type (test) in the member intConstant. • LRT_ALLOCATESTRING must have the string that will be allocated in member strConstant. Any additional parameters for the LRT instructions are popped from the stack.
POP	pop	Pops the value(s) from the stack. The p1 member tells the instruction how many values will be popped. This instruction does not store the popped value any other location. The operation is just adjusting the stack pointer as requested.

Appendix 3: Language Runtime

The functions in the language runtime are not directly called by the RMLEval call operator. These are the functions are called implicitly to implement related operators and memory management. These operators are given in the rmlsup.h between the comments // LRT Start and // LRT End. A short description for each function is also given. Each description starts with index value of the function.

```
// LRT Start
```

```
// 0 (LRT_RESETSCANNER): Reset scanner by space id.
```

```
static uint64_t resetScanner(RMLEval *rmlEval, int64_t selectId);
```

```
// 1 (LRT_ADVANCESCANNER): Advance scanner by space id.
```

```
static uint64_t advanceScanner(RMLEval *rmlEval, int64_t selectId);
```

```
// 2 (LRT_AGGREGATE): Aggregate columns by using select id and wordIndex.
```

```
static uint64_t aggregate(RMLEval *rmlEval, uint64_t selectId);
```

```
// 3 (LRT_SNAPSHOT): Get column values by using select id, and wordIndex.
```

```
static void snapshot(RMLEval *rmlEval, uint64_t selectId);
```

```
// 4 (LRT_SETWORDBASE): Informs the rmlEval about the base address of the temporary words.
```

```
static void setWordBase(RMLEval *rmlEval, uint64_t baseAddress);
```

```
// 5 (LRT_CREATERESULTSET): Create result set by select id.
```

```
static void createResultSet(RMLEval *rmlEval, uint64_t selectId);
```

```
// 6 (LRT_FINALIZERESULTSET): Finalize result set by select id.
static uint64_t finalizeResultSet(RMLEval *rmlEval, uint64_t selectId);

// 7 (LRT_NUMBERTOSTR): Converts a number to string.
static string *number2str(RMLEval *rmlEval, double n);

// 8 (LRT_STRCMP): Compares two strings and returns the test result as true or false.
// test parameter encoding: 0->EQ, 1:>NEQ, 2->GT, 3->LT, 4->GTE, 5->LTE
static int64_t strcmp(string *s1, string *s2, int64_t test);

// 9 (LRT_CATSTRING): Concatenates two strings and returns the concatenated string.
static string *catstring(RMLEval *rmlEval, string *s1, string *s2);

// 10 (LRT_ALLOCATESTRING): Allocates a string given a string constant detected in source.
static string *allocatestring(RMLEval *rmlEval, string *s1);

// 11 (LRT_POSTEXECUTECLEANUP): Performs clean-up after execution. This must be called before the
final return instruction.
static int64_t postexecutecleanup(RMLEval *rmlEval);

// 12 (LRT_CHECKEXCEPTION): Checks for an exception and returns true if an exception record was
generated.
static int64_t checkexception(RMLEval *rmlEval);

// LRT End
```